Introduction to Client-Server Architecture

1. Overview of client-server architecture.

Client–Server architecture is a network model where a **client** requests services and a **server** provides them over a network.

Client

- Sends requests to the server
- Displays the server's response
- Example: Google Chrome

Server

- Receives and processes client requests
- Manages data and resources
- Sends responses back to clients
- Example: Apache HTTP Server

How It Works

1. Client sends request
2. Server processes request
3. Server sends response

Advantages

- Centralized data control
- Better security

- Easy maintenance

Disadvantages

- Server failure affects all clients
- Depends on network

In Java, it is implemented using Sockets (Socket, ServerSocket) and web technologies like Servlets.

2. Difference between client-side and server-side processing.

Client-Side Processing	Server-Side Processing
Performed on the user's computer (browser).	Performed on the server.
Faster response (no server communication needed after load).	Requires communication with server.
Used for UI design and basic validations.	Used for business logic and database operations.
Less secure (code is visible to user).	More secure (code is hidden from user).
Example: JavaScript in Google Chrome	Example: Java Servlets running on Apache Tomcat

Summary:

- **Client-side** = Runs in browser, handles interface and quick validation.
- **Server-side** = Runs on server, handles data processing and security.

3. Roles of a client, server, and communication protocols.

Roles of Client, Server, and Communication Protocols

1. Client

- Sends requests to the server
- Initiates communication
- Displays the response to the user
- Example: A browser like Google Chrome

2. Server

- Receives and processes client requests
- Manages data and resources
- Sends responses back to clients
- Example: Web server like Apache HTTP Server

3. Communication Protocols

- Rules that define how data is transmitted between client and server
- Ensure reliable and secure communication
- Common protocols:
    - HTTP – Used for web communication
    - TCP/IP – Ensures reliable data transmission

HTTP Protocol Overview with Request and Response Headers

1. Introduction to the HTTP protocol and its role in web communication.

Introduction to HTTP Protocol and Its Role in Web Communication

HTTP (HyperText Transfer Protocol) is an application-layer protocol used for communication between a **web client (browser)** and a **web server**.

It is the foundation of data communication on the World Wide Web.

How HTTP Works

1. Client (browser) sends an **HTTP request** to the server.
2. Server processes the request.

3. Server sends back an **HTTP response**.

Example: When you open a website in Google Chrome, it sends an HTTP request to the web server.

Features of HTTP

- **Stateless protocol** (each request is independent)
- Works on **request–response model**
- Uses default port **80**

Role in Web Communication

- Transfers web pages (HTML)
- Sends and receives data between client and server
- Enables communication for web applications

2. Explanation of HTTP request and response headers.

HTTP headers are additional pieces of information sent between the client and server during communication. They provide details about the request or response.

---

1. HTTP Request Headers

These headers are sent by the **client (browser)** to the server.

Purpose:

- Provide information about the client
- Specify the type of data accepted
- Send authentication or cookies

Common Request Headers:

- **Host** – Domain name of the server
- **User-Agent** – Information about the browser (e.g., Google Chrome)
- **Accept** – Type of content the client can handle
- **Authorization** – Credentials for authentication
- **Cookie** – Sends stored cookies to server

---

2. HTTP Response Headers

These headers are sent by the **server** back to the client.

Purpose:

- Provide information about the server
- Describe the returned content
- Control caching and security

Common Response Headers:

- **Content-Type** – Type of content (HTML, JSON, etc.)
- **Content-Length** – Size of the response body
- **Set-Cookie** – Sends cookies to client
- **Server** – Information about server software
- **Cache-Control** – Defines caching rules

J2EE Architecture Overview

1. Introduction to J2EE and its multi-tier architecture.

Java 2 Platform, Enterprise Edition (J2EE) is a platform for developing **enterprise-level, distributed web applications** using Java.

It provides:

- Standard APIs
- Components for web and enterprise applications
- Support for large-scale, secure, and scalable systems

(Modern name: **Jakarta EE**)

Multi-Tier Architecture in J2EE

J2EE follows a **multi-tier (n-tier) architecture**, which divides an application into layers.

1. Client Tier

- User interface
- Sends requests to server
- Example: Web browser

2. Web Tier

- Handles HTTP requests and responses
- Uses Servlets and JSP
- Runs on servers like Apache Tomcat

3. Business Logic Tier

- Contains business rules and processing
- Uses Enterprise JavaBeans (EJB)

4. Data Tier

- Manages database operations
- Stores application data

Advantages of Multi-Tier Architecture

- Better security
- Easy maintenance
- Scalability
- Clear separation of responsibilities

2. Role of web containers, application servers, and database servers.

1. Web Container

A **web container** manages web components like Servlets and JSP.

Role:

- Handles HTTP requests and responses
- Manages servlet life cycle
- Provides security and session management

Example: Apache Tomcat

---

2. Application Server

An **application server** provides a complete environment to run enterprise applications.

Role:

- Executes business logic
- Manages transactions and security
- Supports Enterprise JavaBeans (EJB)
- Integrates web and database services

Example: WildFly

---

3. Database Server

A **database server** stores and manages application data.

Role:

- Stores data securely
- Processes queries
- Ensures data consistency

Example: MySQL

Web Component Development in Java (CGI Programming)

1. Introduction to CGI (Common Gateway Interface).

Common Gateway Interface (CGI) is a standard method used by web servers to generate dynamic web content.

It allows a web server to execute external programs and send their output to the client (browser).

How CGI Works

1. Client sends a request to the web server.
2. Server runs a CGI program (script).

3. The program processes the request (may access database).
4. Output is sent back to the browser as a web page.

Features of CGI

- Generates dynamic content
- Language independent (can be written in C, C++, Perl, etc.)
- Each request starts a new process

Disadvantage

- Slower performance (creates a new process for every request)

2. Process, advantages, and disadvantages of CGI programming.

1. Process of CGI Programming

1. **Client Request:** User submits a form or requests a page in the browser.
2. **Server Invocation:** Web server receives the request and invokes the CGI program.

3. **Processing:** CGI program executes the logic (can access databases or perform calculations).
4. **Response Generation:** Program generates output (usually HTML).
5. **Server Response:** Web server sends the output back to the client's browser.

---

2. Advantages of CGI

- **Language Independent:** Can be written in Perl, C, C++, Java, Python, etc.
- **Dynamic Content:** Generates web pages dynamically based on user input.
- **Standardized:** Supported by most web servers.
- **Simple Concept:** Easy to implement for small tasks.

---

3. Disadvantages of CGI

- **Performance Issues:** Each request creates a new process, slowing down the server.
- **Scalability Problems:** High traffic can overload the server.
- **Security Risks:** Poorly written CGI scripts can be exploited.

- **Resource Intensive:** Uses more CPU and memory compared to modern alternatives like servlets.

Servlet Programming: Introduction, Advantages, and Disadvantages

1. Introduction to servlets and how they work.

Java Servlets are Java programs that run on a **web server** to handle client requests and generate dynamic web content. They are the modern replacement for CGI scripts in Java.

How Servlets Work

1. **Client Request:** Browser sends an HTTP request to the server.
2. **Web Container:** The servlet is managed by a **web container** (e.g., Apache Tomcat) that handles the request lifecycle.
3. **Servlet Processing:** Servlet processes the request (business logic, database access, etc.).
4. **Response Generation:** Servlet generates an HTTP response (HTML, JSON, etc.).
5. **Send to Client:** Web container sends the response back to the browser.

Features of Servlets

- **Platform independent:** Runs anywhere Java is supported.
- **Efficient:** Single servlet instance handles multiple requests using threads.
- **Secure:** Runs inside web container with built-in security.
- **Extensible:** Can integrate with databases, JSP, and frameworks like Spring.

2. Advantages and disadvantages compared to other web technologies.

Advantages

1. **Better Performance than CGI**
    - Single servlet instance handles multiple requests using threads, unlike CGI which creates a new process per request.
2. **Platform Independent**
    - Written in Java, runs on any system with a JVM.
3. **Secure**

- Runs inside a web container with built-in security and session management.

4. **Extensible and Maintainable**
    - Easy to integrate with databases, JSP, and frameworks like Spring Boot.
5. **Robust**
    - Benefits from Java features like exception handling and multithreading.

Disadvantages

1. **Requires Java Knowledge**
    - Developers must know Java to write servlets.
2. **Not Ideal for UI Design**
    - Servlets generate HTML via code; designing complex interfaces is cumbersome. JSP or front-end frameworks are better for UI.
3. **Server Dependency**
    - Needs a servlet container like Apache Tomcat or application server to run.

Servlet Versions, Types of Servlets

1. History of servlet versions.

Version	Release Year	Key Features
Servlet 1.0	1997	Basic API for handling HTTP requests and responses.
Servlet 2.0	1998	Added **HTTP session management** and improved web integration.
Servlet 2.1	1999	Introduced **ServletContext** for application-wide data sharing.
Servlet 2.2	2000	Support for **Java EE integration** and deployment descriptors (web.xml).
Servlet 2.3	2001	Introduced **filters**, **listeners**, and improved **event handling**.
Servlet 2.4	2003	XML-based deployment descriptors enhanced, better JSP integration.
Servlet 2.5	2005	Aligned with **Java EE 5**, added annotations for easier configuration.
Servlet 3.0	2009	Introduced **asynchronous processing**, annotations, and pluggability.

Version	Release Year	Key Features
Servlet 3.1	2013	Added **non-blocking I/O** for scalable web applications.
Servlet 4.0	2017	Support for **HTTP/2**, modern web standards, and improved performance.

2. Types of servlets: Generic and HTTP servlets.

1. Generic Servlet

- **Superclass:** javax.servlet.GenericServlet
- **Protocol-independent:** Can handle any type of protocol (not just HTTP)
- **Features:**
  - Provides basic life cycle methods: init(), service(), destroy()
  - Must override service() method to handle requests
- **Use Case:** Rarely used in modern web apps; mostly for learning or non-HTTP protocols

---

2. HTTP Servlet

- **Superclass:** javax.servlet.http.HttpServlet (extends GenericServlet)
- **Protocol-specific:** Designed for **HTTP requests**
- **Features:**
  - Provides methods like doGet(), doPost(), doPut(), doDelete() for different HTTP methods
  - Handles web-specific tasks easily (sessions, cookies, headers)
- **Use Case:** Most widely used in web applications

Difference between HTTP Servlet and Generic Servlet

1. Detailed comparison between HttpServlet and GenericServlet.

Feature	GenericServlet	HttpServlet
Superclass	javax.servlet.GenericServlet	javax.servlet.http.HttpServlet (extends GenericServlet)
Protocol Support	Protocol-independent (can be used for any protocol)	Protocol-specific (designed for HTTP)

Feature	GenericServlet	HttpServlet
Main Method to Handle Requests	Must override service(ServletRequest req, ServletResponse res)	Provides doGet(), doPost(), doPut(), doDelete() methods for HTTP requests
Usage	Rarely used; for general servlet purposes or learning	Widely used in web applications
Session Management	Not provided by default	Supports HTTP sessions, cookies, and headers
Ease of Web Development	Less convenient for web tasks	Easier and more efficient for handling web requests
Request & Response Objects	Uses ServletRequest and ServletResponse	Uses HttpServletRequest and HttpServletResponse with HTTP-specific methods
Typical Scenario	For custom protocols or advanced use	For dynamic web applications on the internet

Servlet Life Cycle

1. Explanation of the servlet life cycle: init(), service(), and destroy() methods.

A **servlet** goes through a well-defined **life cycle** managed by the web container (e.g., Apache Tomcat).

---

1. init() Method

- Called **once** when the servlet is first loaded into memory.
- Purpose: Perform **initialization tasks** like setting up resources (database connections, configuration).
- Signature:

public void init() throws ServletException

- Example: Loading configuration parameters from web.xml.

---

2. service() Method

- Called **every time a client request** is received.
- Purpose: **Processes requests** and generates responses.
- Signature in GenericServlet:

public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException

- In HttpServlet, this method is usually **overridden by doGet(), doPost(), etc.** for HTTP requests.

---

3. destroy() Method

- Called **once** when the servlet is removed from memory (server shutdown or redeployment).
- Purpose: Release resources like closing database connections or cleaning up memory.
- Signature:

public void destroy()

Creating Servlets and Servlet Entry in web.xml

1. How to create servlets and configure them using web.xml.

1. Creating a Servlet

A servlet is a Java class that extends **HttpServlet** and overrides methods like doGet() or doPost().

Example:

import java.io.*;

```
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloServlet extends HttpServlet {
   public void doGet(HttpServletRequest request, HttpServletResponse response)
         throws ServletException, IOException {

      response.setContentType("text/html");
      PrintWriter out = response.getWriter();
      out.println("<h1>Hello, World!</h1>");
   }
}
```

---

2. Compiling the Servlet

- Save the file as HelloServlet.java
- Compile using javac:

```
javac -classpath <tomcat_lib>/servlet-api.jar HelloServlet.java
```

- Place the .class file in WEB-INF/classes folder of your web application.

---

3. Configuring Servlet in web.xml

web.xml is the **deployment descriptor** that tells the server about the servlet.

Example web.xml Configuration:

```
<web-app>
  <!-- Servlet Declaration -->
  <servlet>
    <servlet-name>HelloServlet</servlet-name>
    <servlet-class>HelloServlet</servlet-class>
  </servlet>

  <!-- Servlet Mapping -->
  <servlet-mapping>
    <servlet-name>HelloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>
</web-app>
```

- <servlet> → Declares the servlet class.
- <servlet-mapping> → Maps the servlet to a URL (/hello).

---

4. Accessing the Servlet

- Start the web server (e.g., Apache Tomcat)
- Open browser and visit:

http://localhost:8080/<your-app-name>/hello

Logical URL and ServletConfig Interface

1. Explanation of logical URLs and their use in servlets.

1. What is a Logical URL?

A **logical URL** is the **URL pattern defined in web.xml** (or via annotations) that maps to a servlet, rather than the actual file path of the servlet class.

- It **hides the physical location** of the servlet class from the client.
- Provides **flexibility** to change servlet class names or locations without affecting the URL used by clients.

---

2. How Logical URLs Work

- In web.xml, you map a servlet to a URL pattern:

<servlet>

```
  <servlet-name>HelloServlet</servlet-name>
  <servlet-class>HelloServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>HelloServlet</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

- Here, /hello is the **logical URL**.
- The client accesses the servlet using:

http://localhost:8080/YourApp/hello

- The server internally forwards the request to HelloServlet.class.

---

3. Advantages of Logical URLs

1. **Encapsulation:** Client doesn't know the actual class name or location.
2. **Flexibility:** Servlet class or package can change without changing the URL.
3. **Consistency:** Multiple URLs can point to the same servlet if needed.

4. **Maintainability:** Easy to update servlet mappings without changing client requests.

2. Overview of ServletConfig and its methods.

ServletConfig is an interface in Java Servlets that provides **configuration information** for a servlet. It is created by the **web container** when the servlet is initialized and allows the servlet to access **initialization parameters** defined in web.xml.

---

1. Purpose of ServletConfig

- Passes **initialization parameters** to a servlet.
- Allows a servlet to get its **servlet name** and **context**.
- Provides a way to configure servlets without hardcoding values.

---

2. Key Methods of ServletConfig

Method	Description
getInitParameter(String name)	Returns the value of a specific initialization parameter defined in web.xml.

Method	Description
getInitParameterNames()	Returns an Enumeration of all initialization parameter names.
getServletName()	Returns the name of the servlet as defined in web.xml.
getServletContext()	Returns the ServletContext object, which allows access to application-wide resources and attributes.

---

3. Example Usage

web.xml initialization parameters:

```
<servlet>
  <servlet-name>ConfigServlet</servlet-name>
  <servlet-class>ConfigServlet</servlet-class>
  <init-param>
    <param-name>adminEmail</param-name>
    <param-value>admin@example.com</param-value>
  </init-param>
</servlet>
```

Servlet code to access ServletConfig:

```
public class ConfigServlet extends HttpServlet {
   public void init() {
      ServletConfig config = getServletConfig();
      String email = config.getInitParameter("adminEmail");
      System.out.println("Admin Email: " + email);
   }
}
```

RequestDispatcher Interface: Forward and Include Methods

1. Explanation of RequestDispatcher and the forward() and include() methods.

RequestDispatcher is an interface in Java Servlets that allows a servlet to **delegate or include requests** to other resources (servlets, JSPs, or HTML pages) on the server.

It is commonly used for **request forwarding** and **content inclusion** within a web application.

---

1. Obtaining a RequestDispatcher

You can get a RequestDispatcher object from the **ServletContext** or **ServletRequest**:

```
// Using ServletContext
RequestDispatcher rd =
getServletContext().getRequestDispatcher("/target.jsp");


// Using ServletRequest
RequestDispatcher rd =
request.getRequestDispatcher("/target.jsp");
```

---

2. Methods of RequestDispatcher

Method	Description
forward(request, response)	Forwards the request from one servlet/JSP to another resource. The client **does not see** the change in URL. The original request and response objects are passed along.
include(request, response)	Includes the content of another resource in the response. The original servlet's output is preserved, and the included resource's output is appended.

---

3. Example: Forwarding

```
// In FirstServlet.java
```

```
RequestDispatcher rd =
request.getRequestDispatcher("/SecondServlet");

rd.forward(request, response);
```

- The request is forwarded to SecondServlet.
- URL in browser remains unchanged.

---

4. Example: Including

```
// In MainServlet.java

RequestDispatcher rd =
request.getRequestDispatcher("/header.jsp");

rd.include(request, response);
```

- Content of header.jsp is included in the response of MainServlet.
- Useful for **common headers, footers, or menus**.

ServletContext Interface and Web Application Listener

1. Introduction to ServletContext and its scope.

ServletContext is a global object that allows **all servlets and JSPs** in a web application to share data and resources.

Key Points:

- **Scope:** Application-wide (accessible to all servlets in the app).
- **Purpose:** Share data, access resources, manage application-level parameters.
- **Common Methods:**
  - setAttribute(name, value) – store data
  - getAttribute(name) – retrieve data
  - getInitParameter(name) – get context param
  - getRequestDispatcher(path) – forward/include requests

Example:

```
getServletContext().setAttribute("appName",
"MyWebApp");

String appName = (String)
getServletContext().getAttribute("appName");
```

2. How to use web application listeners for lifecycle events.

Listeners monitor lifecycle events in a web application (context, session, request).

Key Points:

- **Common Types:**

    - ServletContextListener – app start/stop
    - HttpSessionListener – session create/destroy
    - ServletRequestListener – request lifecycle
- **Usage:** Implement listener interface and override methods.
- **Configure in web.xml:**

```
<listener>
  <listener-class>AppListener</listener-class>
</listener>
```

Java Filters: Introduction and Filter Life Cycle

1. What are filters in Java and when are they needed?

- **Definition:** Java classes that **intercept requests/responses** before reaching a servlet.
- **Purpose:** Preprocess requests or postprocess responses.
- **Common Uses:** Authentication, logging, input validation, compression.
- **Example Flow:**
  Client → Filter → Servlet → Filter → Client

2. Filter lifecycle and how to configure them in web.xml.

1. Filter Lifecycle

1. **init()** – Called once when the filter is first loaded; used for initialization.
2. **doFilter()** – Called for every request; processes or modifies request/response.
3. **destroy()** – Called once when the filter is removed; used to release resources.

---

2. Configuring Filter in web.xml

```
<filter>
  <filter-name>AuthFilter</filter-name>
  <filter-class>AuthFilter</filter-class>
</filter>


<filter-mapping>
  <filter-name>AuthFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>
```

JSP Basics: JSTL, Custom Tags, Scriplets, and Implicit Objects

1. Introduction to JSP and its key components: JSTL, custom tags, scriplets, and implicit objects.

JavaServer Pages (JSP) is a Java technology for creating **dynamic web pages** by embedding Java code in HTML.

Key Components:

1. **JSTL (JavaServer Pages Standard Tag Library)**
    - Predefined tags for common tasks: loops, conditions, database access.
    - Reduces Java code in JSP.
2. **Custom Tags**
    - User-defined tags to encapsulate reusable functionality.
    - Improves modularity and readability.
3. **Scriplets**
    - Java code embedded in JSP using <% ... %>.
    - Example: <% out.println("Hello"); %>
    - Not recommended in modern JSP development.
4. **Implicit Objects**

- Predefined objects available in JSP:
  - request, response, session, application, out, config, pageContext, page, exception

Session Management and Cookies

1. Overview of session management techniques: cookies, hidden form fields, URL rewriting, and sessions.

1. Cookies

- Small data stored on client's browser.
- Tracks user between requests.
- Example: JSESSIONID in Java.

2. Hidden Form Fields

- Include data in <input type="hidden"> in forms.
- Passed with each form submission.

3. URL Rewriting

- Append session info to URL as a query string.
- Example: http://example.com/page?jsessionid=12345

4. HttpSession

- Server-side storage of user-specific data.

- More secure and reliable than cookies.
- Example:

```
HttpSession session = request.getSession();
session.setAttribute("username", "Alice");
```

2. How to track user sessions in web applications.

User sessions can be tracked using these methods:

1. **Cookies** – Store a unique session ID in the client's browser.
2. **URL Rewriting** – Append session ID to URLs when cookies are disabled.
3. **Hidden Form Fields** – Pass session data through forms.
4. **HttpSession (Server-Side)** – Store user data on the server; accessed via request.getSession().

Module-9: Software Design Patterns and Project (MVC + DAO)

1. Definition and purpose of design patterns.

Definition:
A **design pattern** is a **reusable solution** to a common software design problem in a particular context.

Purpose:

- Promote **code reusability** and **maintainability**
- Provide **standardized solutions** for recurring problems
- Improve **communication** among developers
- Enhance **flexibility and scalability** of software

2. Classification: Creational, Structural, and Behavioural patterns.

1. **Creational Patterns**
    - Deal with **object creation** mechanisms.
    - Examples: Singleton, Factory, Builder, Prototype.
2. **Structural Patterns**
    - Deal with **object composition** and relationships.

    - Examples: Adapter, Decorator, Facade, Composite.
3. **Behavioural Patterns**
    - Deal with **object interaction and communication**.
    - Examples: Observer, Strategy, Command, Template Method.

3. Examples of popular patterns: Singleton, Factory, Observer, Decorator, etc.

1. **Singleton** – Ensures a class has **only one instance**.
2. **Factory** – Creates objects without exposing the **instantiation logic**.
3. **Observer** – Defines a **one-to-many dependency**; observers update automatically when the subject changes.
4. **Decorator** – Adds **new behavior** to objects dynamically without changing their code.
5. **Strategy** – Encapsulates **algorithms** and makes them interchangeable.
6. **Facade** – Provides a **simplified interface** to a complex system.

4. Model-View-Controller (MVC) architecture explained.

MVC is a design pattern that **separates an application** into three components for better organization and maintainability.

---

1. Model

- Represents **data and business logic**.
- Manages database interactions and application state.

2. View

- Handles **presentation/UI**.
- Displays data from the model to the user.

3. Controller

- Manages **user input** and updates the model or view accordingly.
- Acts as a **bridge** between model and view.

Flow:
User → Controller → Model → View → User

5. Separation of concerns and how MVC helps in structuring applications.

Separation of Concerns (SoC):

- Dividing an application into **distinct parts**, each handling a specific responsibility.
- Makes code **easier to maintain, test, and scale**.

How MVC Helps:

- **Model:** Handles business logic and data (no UI code).
- **View:** Handles presentation (no business logic).
- **Controller:** Handles user input and coordinates between Model and View.

6. Purpose of the DAO pattern in decoupling data access logic from business logic.

DAO (Data Access Object) Pattern is a design pattern that **separates the data access logic from the business logic** in an application.

Key Points:

1. **Decoupling:**
    - Business logic doesn't directly interact with the database.

- Changes in database or storage mechanism do not affect business code.

2. **Reusability:**
   - Data access methods (like CRUD operations) can be reused across different parts of the application.
3. **Maintainability:**
   - Makes the code cleaner and easier to maintain.
   - Fixing or updating database-related code doesn't impact business logic.
4. **Testability:**
   - Easier to write unit tests for business logic by mocking DAO classes.

Example Flow:

Business Layer → DAO Layer → Database

7. How DAO works in combination with MVC to interact with databases.

In a **MVC (Model-View-Controller)** architecture, the **DAO pattern** helps manage database interactions while keeping the layers separate.

---

1. Role in MVC

- **Model:** Represents the application data and business logic.
- **Controller:** Receives user requests, interacts with the Model.
- **View:** Displays data to the user.
- **DAO:** Handles all **database operations** for the Model.

---

2. Workflow

1. **User Request:** Sent from **View** to **Controller**.
2. **Controller:** Calls methods in the **Model**.
3. **Model:** Uses **DAO** to fetch or update data in the database.
4. **DAO:** Executes SQL queries and returns results to the Model.
5. **Controller:** Updates the **View** with processed data.

---

3. Benefits

- Keeps **database code separate** from business logic (Model).
- Promotes **code reuse** and **maintainability**.

Session Management (Session, Cookie, Hidden Form Field, URL Rewriting)

1. Why session management is essential in web applications.

Session management is crucial because **HTTP is a stateless protocol**, meaning the server does not remember previous requests from the same user. Without session management, each request would be treated as a new user interaction.

Key Reasons:

1. **User Identification:**
    - Tracks logged-in users across multiple requests.
2. **Personalization:**
    - Maintains user preferences, shopping carts, or form data.
3. **Security:**
    - Helps enforce authentication and access control.
4. **State Maintenance:**
    - Preserves the application state, like multi-page forms or transactions.

2. Difference between client-side and server-side session management.

Feature	Client-Side Session	Server-Side Session
Storage	Session data stored on the **client** (browser), usually via cookies or hidden fields	Session data stored on the **server** (e.g., HttpSession)
Security	Less secure; data can be manipulated by the client	More secure; server controls data access
Scalability	Light on server resources	Requires server memory to store sessions
Example	Cookies storing userId or preferences	HttpSession storing user object in server memory
State Maintenance	Limited by client capabilities and cookie size	Reliable and supports complex objects

3. Definition of a session and its importance in tracking user activity.

Definition:
A **session** is a temporary, server-recognized period of interaction between a **user (client)** and a **web application**, during which the server maintains state about the user.

Importance in Tracking User Activity

1. **Maintains State:** Tracks user interactions across multiple HTTP requests.
2. **User Identification:** Recognizes logged-in users and prevents repeated logins.
3. **Personalization:** Stores preferences, shopping carts, or form data for the user.
4. **Security & Access Control:** Associates actions with authenticated users to enforce permissions.

4. How to create, retrieve, and destroy sessions using Java servlets.

1. Creating a Session

- Use request.getSession() to create or retrieve a session.

```
HttpSession session = request.getSession(); // creates new session if none exists

session.setAttribute("username", "Alice");
```

---

2. Retrieving a Session

- Use request.getSession(false) to get an existing session without creating a new one.

```
HttpSession session = request.getSession(false);

if (session != null) {

   String user = (String) session.getAttribute("username");

}
```

---

3. Destroying a Session

- Use invalidate() to terminate the session.

```
HttpSession session = request.getSession(false);

if (session != null) {

   session.invalidate(); // removes session and all attributes

}
```

5. What cookies are and how they store small amounts of data on the client-side.

Definition:
A **cookie** is a small piece of data sent by a web server and stored on the **client's browser** to maintain state and track user activity.

How Cookies Work

1. Server creates a cookie and sends it in the **HTTP response**:

Set-Cookie: username=Alice; Max-Age=3600; Path=/

2. Browser stores the cookie locally and sends it back in **subsequent requests**:

Cookie: username=Alice

Key Points

- Stores **small amounts of data** (typically up to 4 KB).
- Can have attributes like **expiry time, path, and domain**.
- Used for **session tracking, personalization, and preferences**.

6. Creating, reading, updating, and deleting cookies in Java servlets.

1. Creating a Cookie

```
Cookie cookie = new Cookie("username", "Alice");
cookie.setMaxAge(3600); // expires in 1 hour
response.addCookie(cookie);
```

---

2. Reading Cookies

```
Cookie[] cookies = request.getCookies();
if (cookies != null) {
   for (Cookie c : cookies) {
      if (c.getName().equals("username")) {
         String value = c.getValue();
      }
   }
}
```

---

3. Updating a Cookie

- Create a cookie with the **same name** and set a new value:

```
Cookie cookie = new Cookie("username", "Bob");
```

```
cookie.setMaxAge(3600);
response.addCookie(cookie);
```

---

4. Deleting a Cookie

- Set **max age to 0** and send it to the client:

```
Cookie cookie = new Cookie("username", "");
cookie.setMaxAge(0);
response.addCookie(cookie);
```

7. Explanation of hidden form fields and their role in passing data between pages.

Definition:
Hidden form fields are <input type="hidden"> elements in HTML forms that **store data not visible to the user** but submitted along with the form.

Role in Passing Data

- **Maintain state:** Pass values (like user ID, session info) between pages without displaying them.
- **Supports multi-page forms:** Keeps data from previous steps intact.

- **Alternative to cookies or URL rewriting** when simple data transfer is needed.

Example

```
<form action="NextPageServlet" method="post">
  <input type="hidden" name="userId" value="12345">
  <input type="submit" value="Next">
</form>
```

- On the server:

```
String userId = request.getParameter("userId");
```

8. How URL rewriting can be used to track sessions when cookies are disabled.

URL rewriting is a technique where the **session ID is appended to the URL** when cookies are disabled.

How It Works

- Server adds the session ID as a **query parameter** to every link.
- Example:

http://example.com/home;jsessionid=ABC123

or

http://example.com/home?sessionId=ABC123

- The server reads this ID from the URL to identify the user session.

In Java Servlets

Use response.encodeURL() to automatically append the session ID if needed:

```
String url = response.encodeURL("home.jsp");
```

If cookies are disabled, the session ID is added to the URL automatically.

Importance

- Ensures **session tracking works even without cookies**.
- Commonly used as a **fallback mechanism**.

Template Integration

1. What is template integration in web applications.

Definition:
Template integration is the practice of **separating the static layout (HTML/CSS) from dynamic content** in a web application using templates.

Purpose

- **Reusability:** Common layouts (header, footer, menu) can be reused across pages.
- **Maintainability:** Changes to layout or design are applied automatically to all pages.
- **Separation of Concerns:** Keeps **presentation** separate from **business logic**.

How It Works

- Templates define placeholders for dynamic content.
- Server-side code (JSP, Thymeleaf, or other template engines) fills these placeholders at runtime.

Example:

```
<jsp:include page="header.jsp" />
```

```
<p>Welcome, ${username}</p>

<jsp:include page="footer.jsp" />
```

2. Importance of using pre-built templates for faster UI development.

Importance of Pre-Built Templates for UI Development

1. **Speeds Up Development:**
    - Provides ready-made layouts, components, and styles.
    - Reduces time spent on designing pages from scratch.
2. **Consistency:**
    - Ensures a uniform look and feel across the application.
3. **Reduces Errors:**
    - Templates are usually tested and responsive, minimizing layout issues.
4. **Focus on Business Logic:**
    - Developers can concentrate on **application functionality** instead of UI design.

Image Upload/Download

1. Steps to upload and download files in Java web applications.

1. File Upload

Steps:

1. Create an HTML form with enctype="multipart/form-data":

```
<form action="UploadServlet" method="post" enctype="multipart/form-data">
  <input type="file" name="file">
  <input type="submit" value="Upload">
</form>
```

2. In the servlet, use **Servlet 3.0 API** or libraries like Apache Commons FileUpload:

```
@MultipartConfig
public class UploadServlet extends HttpServlet {
  protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
    Part filePart = request.getPart("file");
    String fileName = filePart.getSubmittedFileName();
    filePart.write("C:/uploads/" + fileName);
```

```
    }
}
```

---

2. File Download

Steps:

1. Set response content type and headers in the servlet:

```
protected void doGet(HttpServletRequest request,
HttpServletResponse response)
      throws ServletException, IOException {
    String filePath = "C:/uploads/sample.txt";
    File file = new File(filePath);

    response.setContentType("application/octet-stream");
    response.setHeader("Content-Disposition",
"attachment;filename=" + file.getName());

    FileInputStream in = new FileInputStream(file);
    ServletOutputStream out =
response.getOutputStream();
    byte[] buffer = new byte[1024];
    int length;
```

```
    while ((length = in.read(buffer)) > 0) {
        out.write(buffer, 0, length);
    }
    in.close();
    out.close();
}
```

2. Explanation of the multipart request and handling file uploads using MultipartConfig.

1. Multipart Request

- A **multipart request** allows a client to **send files along with form data** to the server.
- The form must use:

```
<form action="UploadServlet" method="post" enctype="multipart/form-data">
    <input type="file" name="file">
    <input type="submit" value="Upload">
</form>
```

- enctype="multipart/form-data" tells the server that the request contains **binary data**.

---

2. Handling File Upload in Servlets

- Java Servlet 3.0+ provides the @MultipartConfig annotation.
- This enables easy access to uploaded files via request.getPart().

Example:

```
import javax.servlet.*;

import javax.servlet.annotation.*;

import javax.servlet.http.*;

import java.io.*;


@WebServlet("/UploadServlet")

@MultipartConfig(

   fileSizeThreshold = 1024 * 1024, // 1MB

   maxFileSize = 5 * 1024 * 1024,   // 5MB

   maxRequestSize = 10 * 1024 * 1024 // 10MB

)

public class UploadServlet extends HttpServlet {

   protected void doPost(HttpServletRequest request,
HttpServletResponse response)

         throws ServletException, IOException {
```

```
        Part filePart = request.getPart("file");
        String fileName = filePart.getSubmittedFileName();
        filePart.write("C:/uploads/" + fileName);
    }
}
```

---

3. Key Points

- fileSizeThreshold – Size after which file is written to disk.
- maxFileSize – Maximum size allowed for a single file.
- maxRequestSize – Maximum size allowed for the whole request.
- request.getPart("fieldName") retrieves the uploaded file.

Mail Integration

1. How to send emails from a Java web application using JavaMail API.

1. Add JavaMail Dependency

If using Maven:

```
<dependency>
```

```
    <groupId>com.sun.mail</groupId>
    <artifactId>jakarta.mail</artifactId>
    <version>2.1.1</version>
</dependency>
```

---

2. Set Up Mail Properties

```
Properties props = new Properties();
props.put("mail.smtp.host", "smtp.example.com");
props.put("mail.smtp.port", "587");
props.put("mail.smtp.auth", "true");
props.put("mail.smtp.starttls.enable", "true");
```

---

3. Authenticate

```
Session session = Session.getInstance(props, new
javax.mail.Authenticator() {
    protected PasswordAuthentication
getPasswordAuthentication() {
        return new
PasswordAuthentication("your_email@example.com",
"password");
    }
});
```

---

4. Create and Send Message

```
try {
   Message message = new MimeMessage(session);
   message.setFrom(new
InternetAddress("your_email@example.com"));
   message.setRecipients(Message.RecipientType.TO,
         InternetAddress.parse("recipient@example.com"));
   message.setSubject("Test Email");
   message.setText("Hello, this is a test email from Java
web app.");

   Transport.send(message);
   System.out.println("Email sent successfully!");
} catch (MessagingException e) {
   e.printStackTrace();
}
```

2. Explanation of SMTP and how it's used for sending emails.

SMTP (Simple Mail Transfer Protocol):

- A standard protocol used to **send emails** from a client to a mail server or between servers.
- Works over **TCP port 25, 465, or 587** depending on encryption (TLS/SSL).

How SMTP Works

1. **Client (Application)** connects to the **SMTP server**.
2. Sends email headers (From, To, Subject) and message body.
3. SMTP server **forwards the email** to the recipient's mail server.
4. Recipient server stores the email for retrieval via POP3 or IMAP.

In JavaMail

- JavaMail uses SMTP by setting properties like:

```
props.put("mail.smtp.host", "smtp.example.com");
props.put("mail.smtp.port", "587");
props.put("mail.smtp.auth", "true");
```

```
props.put("mail.smtp.starttls.enable", "true");
```

- Then the application **authenticates** and sends the email using Transport.send().

OTP via Mail Integration

1. Introduction to OTP (One-Time Password) and its importance in enhancing security.

Definition:
An **OTP (One-Time Password)** is a **temporary, single-use password** used to verify a user's identity during authentication or transactions.

Importance in Enhancing Security

1. **Prevents Password Theft:** OTP is valid only once, so stolen passwords cannot be reused.
2. **Two-Factor Authentication (2FA):** Adds an extra layer beyond username/password.
3. **Time-Sensitive:** Usually expires within a few minutes, reducing the window for attacks.
4. **Protects Sensitive Operations:** Used for banking transactions, password resets, and secure logins.

2. How to generate and send OTP via email for verification purposes.

1. Generate OTP

```
import java.util.Random;


public String generateOTP(int length) {
   Random random = new Random();
   StringBuilder otp = new StringBuilder();
   for (int i = 0; i < length; i++) {
      otp.append(random.nextInt(10)); // digits 0-9
   }
   return otp.toString();
}
```

- Generates a numeric OTP of desired length (e.g., 6 digits).

---

2. Send OTP via Email

- Use **JavaMail API**:

```
Properties props = new Properties();
props.put("mail.smtp.host", "smtp.example.com");
props.put("mail.smtp.port", "587");
```

```
props.put("mail.smtp.auth", "true");
props.put("mail.smtp.starttls.enable", "true");

Session session = Session.getInstance(props, new
javax.mail.Authenticator() {
   protected PasswordAuthentication
getPasswordAuthentication() {
      return new
PasswordAuthentication("your_email@example.com",
"password");
   }
});

try {
   Message message = new MimeMessage(session);
   message.setFrom(new
InternetAddress("your_email@example.com"));
   message.setRecipients(Message.RecipientType.TO,
         InternetAddress.parse("user@example.com"));
   message.setSubject("Your OTP Code");
   message.setText("Your OTP is: " + otp);
```

```
    Transport.send(message);
} catch (MessagingException e) {
    e.printStackTrace();
}
```

---

3. Typical Flow

1. Generate OTP.
2. Store OTP temporarily on the server (e.g., in session or database).
3. Send OTP to user via email.
4. User enters OTP → server validates it for verification.

Online Payment Integration

1. Introduction to online payment gateways (e.g., PayPal, Stripe).

Definition:
An **online payment gateway** is a service that **authorizes and processes payments** made over the internet between a customer and a merchant.

Examples

- **PayPal** – Widely used for online transactions and international payments.
- **Stripe** – Provides APIs for integrating card payments into web and mobile apps.

Key Functions

1. **Payment Authorization:** Verifies credit/debit card or digital wallet details.
2. **Secure Transactions:** Encrypts sensitive data (PCI-DSS compliant).
3. **Transaction Processing:** Transfers funds from customer to merchant.
4. **Notifications & Reporting:** Confirms payment success or failure.

2. How to integrate payment gateways into web applications.

1. Choose a Gateway

- Examples: **PayPal, Stripe, Razorpay**.
- Create a **merchant account** and get API credentials (keys).

---

2. Add Gateway SDK or API

- Most gateways provide **Java SDKs or REST APIs** for integration.
- Include the SDK in your project (Maven/Gradle) or use HTTP calls for REST APIs.

---

3. Create Payment Request

- Collect payment details: amount, currency, order ID.
- Send a request to the gateway API to **generate a payment session or token**.

```
// Example pseudo-code for Stripe

Stripe.apiKey = "sk_test_XXXX";

Map<String, Object> params = new HashMap<>();

params.put("amount", 5000); // in cents

params.put("currency", "usd");

params.put("payment_method_types",
Arrays.asList("card"));

PaymentIntent intent = PaymentIntent.create(params);
```

---

4. Redirect User / Collect Payment

- Redirect the user to the gateway's **secure payment page** or embed a payment form.

---

5. Handle Response

- Gateway sends a **success/failure callback** to your application.
- Verify transaction and update your database/order status.

<u>AJAX</u>

1. Introduction to AJAX and its role in improving the user experience by enabling asynchronous requests.

Definition:
AJAX (Asynchronous JavaScript and XML) is a technique that allows web pages to **communicate with the server asynchronously** without reloading the entire page.

Role in Improving User Experience

1. **Partial Page Updates:** Only updates necessary content, making pages faster.
2. **Reduced Bandwidth:** Sends/receives only required data instead of full page reloads.
3. **Responsive UI:** Users can interact with the page while data loads in the background.

4. **Dynamic Applications:** Enables real-time features like search suggestions, notifications, and live data updates.

2. Explanation of how AJAX works in combination with JavaScript and the server.

How AJAX Works with JavaScript and the Server

1. **JavaScript Sends Request:**
   - JavaScript creates an **XMLHttpRequest** or uses fetch() to send data to the server **asynchronously**.
2. **Server Processes Request:**
   - A server-side script (Servlet, JSP, PHP, etc.) receives the request, processes it, and generates a response (often JSON or XML).
3. **JavaScript Receives Response:**
   - JavaScript callback functions handle the server's response and **update the page dynamically** without reloading.

Example (Simplified)

```
fetch('dataServlet')
  .then(response => response.json())
```

```
  .then(data => {
    document.getElementById('result').innerText =
data.message;
  });
```

Flow:
User Action → JavaScript AJAX Request → Server Processing → Response → Update Page

---